

(D0 through D7) of LCD1. PORTC pins PC0 and PC1 of IC4 drive two control pins RS and EN of LCD1, respectively. Control pin R/W of LCD1 is connected to ground. A 1-kilo-ohm preset is connected to pin 3 of LCD1 to control the contrast of the display.

An 8MHz crystal is connected to crystal input pins of ATmega16 along with two ceramic capacitors forming the clock circuitry. Complete circuit operation is due to the software program embedded into the AVR MCU.

Power supply. The circuit requires 5V DC regulated voltage. The power supply circuit is shown in Fig. 2. 5V DC is derived from 230V AC mains through a transformer (X1), bridge rectifier DB107 (IC1), 7805 regulator (IC2) and a few other components. Glowing of LED1 indicates the presence of power in the circuit.

Software

The software program is written in C language. It is compiled using AVR Studio software.

Software logic. Software logic is to find two things: setting the angle of the motor rotation as per position of joystick shaft and positioning the stepper motor in predefined start angle at power on.

As the potentiometer is rotated, analogue voltage input to the ADC pin is varied and so the corresponding digital input also varies. As analogue input varies from 0V to 5V, digital input varies from 0 to 1023 (because ATmega16 has 10-bit ADC resolution). This complete range of 0 to 1023 is divided into 12 different ranges such as 0-85, 86-171, 172-257 and so on. The motor angle is set as per the digital value listed in the table.

Motor rotation directly depends on its step resolution. Motor step resolution is $3.75^\circ/\text{pulse}$. That means, for each pulse applied, motor rotates by 3.75° only. But it is required to give four pulses in a sequence to all four motor coils. So the motor will rotate by $3.75^\circ \times 4 = 15^\circ$. Thus, we can rotate the motor in multiples of 15° such as 15° , 30° , 45° , 60° , 75°

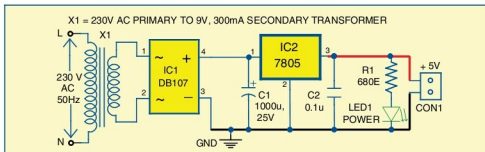


Fig. 2: Circuit of the power supply

PARTS LIST

Semiconductors:

- IC1 - DB107 bridge rectifier
- IC2 - 7805, 5V regulator
- IC3 - ULN2003 driver
- IC4 - ATmega16 MCU
- LED1 - 5mm LED
- LCD1 - 16x2 alphanumeric display module

Resistors (all 1/4-watt, $\pm 5\%$ carbon):

- R1 - 680-ohm
- R2 - 100-ohm
- R3 - 10-kilo-ohm
- VR1 - 1-kilo-ohm preset
- JP1 - 10-kilo-ohm joystick potentiometer

Capacitors:

- C1 - 1000µF, 25V electrolytic
- C2 - 0.1µF ceramic
- C3, C4 - 22pF ceramic
- C5 - 10µF, 16V electrolytic

Miscellaneous:

- X1 - 230V AC primary to 9V, 300mA secondary transformer
- XTAL1 - 8MHz crystal oscillator
- M1 - $3.75^\circ/\text{step}$, 5V stepper motor (unipolar)
- CON1 - 2-pin berg strip male connector
- CON2 - 2-pin terminal connector
- CON3 - 5-pin berg strip male connector
- S1 - 16-pin berg strip female connector for LCD1
- S1 - Tactile switch

Different Angles of Stepper Motor Rotation

Serial number	Range of digital values	Corresponding angles (in degree)
1	0-85	0
2	86-171	30
3	172-257	60
4	258-343	90
5	344-429	120
6	430-515	150
7	516-601	180
8	602-687	210
9	688-773	240
10	774-859	270
11	860-945	300
12	946-1023	330

Number of pulses = $4 \times (75-45)/15 = 8$

Therefore the motor rotates for $8 \times 3.75^\circ = 30^\circ$

If the new set angle value is less than the previous set angle value, the result will be negative and the motor will rotate in reverse direction.

Program functions. The program is made up of different functions as explained below.

`Senddata()` sends one byte data to the LCD on its data bus.

`sendcommand()` sends one byte command to the LCD on its data bus.

`printstr()` displays the message or string on the LCD.

`inc_angle()` increments the angle by 15° and displays on the LCD.

`dec_angle()` decrements the angle by 15° and displays on the LCD.

`display_value()` takes the integer angle value as input, converts all digits into ASCII and displays it as angle value on the LCD.